

Autism FER – সম্পূর্ণ ব্যাখ্যা গাইড

Autism Facial Expression Recognition – সম্পূর্ণ ব্যাখ্যা গাইড

(Supervisor প্রশ্ন করলে যেন সব উত্তর দিতে পারো)

এই ফাইলটি তোমার University Supervisor-এর যেকোনো প্রশ্নের উত্তর দেওয়ার জন্য তৈরি করা হয়েছে। প্রতিটি বিষয় বাংলায় সহজভাবে এবং Technical depth সহ ব্যাখ্যা করা হয়েছে।

সূচিপত্র

1. [প্রজেক্টের মূল উদ্দেশ্য](#)
2. [Dataset](#) — কোথা থেকে, কেন, কতটুকু
3. [Data Cleaning](#) — কেন দরকার হলো
4. [Class Imbalance](#) সমস্যা ও সমাধান
5. [Baseline Models](#) — কেন এগুলো বেছে নেওয়া হয়েছে
6. [Training Setup](#) — Hyperparameters ব্যাখ্যা
7. [Stratified K-Fold Cross-Validation](#) — কেন?
8. [Loss Functions](#) — Focal Loss vs Cross-Entropy
9. [EMA \(Exponential Moving Average\)](#) — কী ও কেন
10. [Proposed Model Architecture](#) — বিস্তারিত
11. [SE Block \(Squeeze-and-Excitation\)](#)
12. [Two-Stage Training](#) — কেন দুটো Stage?
13. [TTA \(Test-Time Augmentation\)](#)
14. [Uncertainty Guardrail](#) — Clinical Safety
15. [Evaluation Metrics](#) — কোনটা কী বোঝায়
16. [Results ও Findings](#)
17. [Grad-CAM](#) — Explainability
18. [কেন MTCNN+CLAHE বাদ দেওয়া হলো](#)
19. [Kaggle এ কেন Run করা হয়](#)
20. [Supervisor এর সম্ভাব্য প্রশ্ন ও উত্তর](#)

১. প্রজেক্টের মূল উদ্দেশ্য

কী করা হচ্ছে?

Autism Spectrum Disorder (ASD) আক্রান্ত শিশুদের মুখের ছবি দেখে তাদের **৬টি Emotion** চেনার একটি Deep Learning System তৈরি করা হচ্ছে।

৬টি Emotion কী কী?

Emotion	বাংলা
Anger	রাগ
Fear	ভয়
Joy	আনন্দ
Natural	স্বাভাবিক/নিরপেক্ষ
Sadness	দুঃখ
Surprise	বিস্ময়

কেন Autism-এর জন্য আলাদা Research?

- ASD শিশুরা সাধারণত facial expression কম প্রকাশ করে অথবা ভিন্নভাবে প্রকাশ করে।
- সাধারণ FER (Facial Expression Recognition) model (যেমন FER2013 দিয়ে trained) এই শিশুদের ক্ষেত্রে কাজ করে না।
- Clinical সেটিংয়ে ASD শিশুর Emotion বোঝা Therapist ও Parents-এর জন্য গুরুত্বপূর্ণ।

গবেষণার অবদান কী? (Research Contribution)

1. **Comparative Study:** ৯-১০টি State-of-the-art Architecture-কে একই fair protocol-এ test করা হয়েছে।
2. **Proposed Architecture:** VGG16 (CNN) + DeiT (Transformer) — দুটি ভিন্ন ধরনের Feature Extractor একসাথে ব্যবহার করা হয়েছে।
3. **Data Cleaning:** Dataset-এর মধ্যে duplicate ও conflicting label সরানো হয়েছে — একটি methodological contribution।
4. **Fair Protocol:** Stratified K-Fold CV ব্যবহার করে statistically valid result দেওয়া হয়েছে।
5. **Explainability:** Grad-CAM দিয়ে দেখানো হয়েছে model কোন অংশ দেখে decision নেয়।

২. Dataset

Dataset কোথা থেকে এলো?

৪টি আলাদা Source থেকে data নেওয়া হয়েছে এবং merge করা হয়েছে:

Source	বিবরণ
FERAC Dataset (Kaggle)	4-class ASD facial expressions
Nora Mahmoud's Mendeley Dataset	ASD/Non-ASD labeled faces
Dr. Fatma M. Talaat (Kaggle)	ASD facial emotion data
Hasibur Rahman's Kaggle Dataset	ASD facial expression samples

কেন Multiple Source?

- Single source থেকে যথেষ্ট data পাওয়া যাচ্ছিল না।
- বিশেষত **Fear** এবং **Surprise** class-এ খুব কম image ছিল।
- Multiple source merge করলে diversity বাড়ে, model generalize করতে পারে ভালো।

Original Dataset vs Cleaned Dataset

	dataset/ (Original)	dataset_clean/ (Cleaned)
Image Count	1,988	1,808
Duplicates	আছে	সরানো হয়েছে
Label Conflicts	আছে	সরানো হয়েছে
Pipeline ব্যবহার করে?	না	হ্যাঁ

Final Dataset Distribution (dataset_clean/):

Class	Count	মোটের %
Anger	167	9.2%
Fear	68	3.8%
Joy	843	46.6%
Natural	201	11.1%
Sadness	404	22.3%
Surprise	125	6.9%
Total	1,808	100%

৩. Data Cleaning

কী কী সমস্যা ছিল Original Dataset-এ?

সমস্যা ১: Near-Duplicate Images

- একই ছবি (বা প্রায় একই ছবি) dataset-এ একাধিকবার ছিল।
- কিছু ছবি train split-এ ছিল এবং একই ছবি test split-এও ছিল।
- এতে **Data Leakage** হয় — model test-এ যাকে “নতুন” ছবি মনে করে, সে আসলে training-এ দেখা ছবি।
- ফলে accuracy artificially বেড়ে যায় — false high result!

সমস্যা ২: Label Conflicts

- একই মুখের ছবি এক জায়গায় “Sadness” আর অন্য জায়গায় “Fear” বলে labeled ছিল।
- এটি model-কে confuse করে।

কীভাবে Clean করা হলো?

- **dHash (Difference Hash)** algorithm ব্যবহার করা হয়েছে।
- dHash একটি image-এর “fingerprint” তৈরি করে।
- দুটি image-এর fingerprint-এর মধ্যে পার্থক্য (Hamming Distance) ≤ 4 হলে সেগুলো “near-duplicate” বলে ধরা হয়।
- **৪৪টি near-duplicate cluster** পাওয়া গেছে।
- **৪৬টি label conflict** সরানো হয়েছে।
- ফলাফল: `cleaning_report.json` ফাইলে সংরক্ষিত আছে।

এটা কেন গুরুত্বপূর্ণ?

“Garbage in, Garbage out” — dirty data দিয়ে train করলে model অকার্যকর হয় এবং artificially high result পাওয়া যায় যা publication-এ dishonest।

8. Class Imbalance

সমস্যাটা কী?

Dataset-এ Joy = 843 কিন্তু Fear = 68। অনুপাত প্রায় **12:1**।

যদি model কে বলা “এই 1808টা ছবি দেখে Emotion বলো” — model শিখবে সবসময় “Joy” বলতে। তাহলেও 46% accuracy পাবে। কিন্তু Fear, Surprise কখনো ঠিকমতো চিনবে না।

সমাধান ১: WeightedRandomSampler (Baseline Models)

- Training-এর সময় minority class (Fear, Surprise) থেকে বেশি বার sample নেওয়া হয়।
- প্রতিটি sample-এর $\text{weight} = 1 / \text{class_count}$
- এতে করে model প্রতিটি class-কে সমান গুরুত্ব দিয়ে দেখে।

```
counts = Counter(train_ds.labels)
sample_weights = [1.0 / counts[label] for label in train_ds.labels]
sampler = WeightedRandomSampler(sample_weights, len(train_ds), replacement=True)
```

সমাধান ২: Focal Loss Alpha (Proposed Model)

- Proposed Model-এ Focal Loss-এ alpha parameter ব্যবহার করা হয়।
- Alpha = inverse frequency (বিরল class → বেশি weight)।
- অতিরিক্ত boost: Sadness $\times 2.0$, Fear $\times 1.2$ ।

```
alpha[CLASSES.index("sadness")] *= 2.0
alpha[CLASSES.index("fear")] *= 1.2
```

কেন দুটো একসাথে ব্যবহার করা হয়নি?

- **Double-weighting bug:** পুরনো version-এ Sampler + Class Weight দুটোই ব্যবহার করা হয়েছিল।
- এতে model minority class-কে অতিরিক্ত penalize করে, small model collapse করে।
- Fix: Baseline-এ শুধু Sampler, Proposed-এ শুধু Focal Loss Alpha।

৫. Baseline Models

কোন কোন Model তুলনা করা হয়েছে?

Model	ধরন	Parameters	বিশেষত্ব
VGG-16 (BN)	CNN	134.3M	Classic, powerful feature extractor
InceptionV3	CNN	21.8M	Multi-scale convolutions, 299 input
DenseNet-121	CNN	7.0M	Dense connections, feature reuse
EfficientNet-B0	CNN	~5M	Compound scaling, efficient
MobileNetV2	CNN	2.2M	Lightweight, mobile-friendly
ResNet-50	CNN	23.5M	Skip connections, residual learning
DeiT-Small	Transformer	22.1M	Knowledge distillation, no CNN
ViT-Base	Transformer	85.8M	Pure attention, ImageNet-21k
Swin-Base	Hybrid	86.7M	Shifted window attention
Swin-Tiny	Hybrid	~28M	Smaller Swin variant

CNN কীভাবে কাজ করে?

- ছবিকে ছোট ছোট patch-এ কেটে filter দিয়ে scan করে।
- প্রতিটি filter একটি pattern খোঁজে (edge, texture, shape)।
- অনেকগুলো layer পার হলে high-level feature (চোখ, নাক, মুখ) বের হয়।
- VGG16 এর 13টি Convolutional Layer আছে।

Vision Transformer (ViT) কীভাবে কাজ করে?

- ছবিকে 16×16 pixel-এর patch-এ ভাগ করে।
- প্রতিটি patch একটি “token” — ঠিক NLP-তে word-এর মতো।
- Self-Attention mechanism দিয়ে প্রতিটি patch অন্য সব patch-এর সাথে interact করে।

- Global context ভালো বোঝে, কিন্তু local texture-এ CNN-এর চেয়ে দুর্বল।

DeiT কী আলাদা?

- DeiT = Data-efficient Image Transformer।
- ViT-এর মতো কিন্তু ImageNet-21k ছাড়াই ImageNet-1k দিয়ে train করা।
- Knowledge Distillation ব্যবহার করে — একটি CNN teacher থেকে শিখেছে।
- ছোট dataset-এ ViT-এর চেয়ে ভালো কাজ করে।

Swin Transformer কী?

- Shifted Window Transformer।
- ছবিকে window-এ ভাগ করে attention calculate করে — CNN-এর locality ধরে রাখে।
- CNN এবং Transformer-এর hybrid benefit পায়।

কেন Differential Learning Rate ব্যবহার করা হয়েছে?

Backbone = pretrained features → ছোট LR দিয়ে ধীরে update
backbone LR = $lr \times 0.1$

Head = নতুন classification layer → বড় LR দিয়ে দ্রুত শিখাবে
head LR = $lr \times 1.0$

- Pretrained features নষ্ট না করে শুধু classifier adapt করা।
- CNN-এর জন্য: $lr = 1e-3$
- Transformer-এর জন্য: $lr = 1e-4$ (বেশি LR দিলে collapse করে)

৬. Training Setup

Augmentation কী ও কেন?

Training-এর সময় প্রতিটি image-কে random transformation দেওয়া হয়:

<code>transforms.RandomHorizontalFlip(p=0.5)</code>	# আয়নায় উল্টে দেখা
<code>transforms.RandomRotation(10)</code>	# ± 10 degree ঘোরানো
<code>transforms.RandomAffine(translate=(0.1, 0.1))</code>	# একটু সরানো
<code>transforms.ColorJitter(brightness=0.2)</code>	# আলো-অন্ধকার বদলানো
<code>transforms.RandomGrayscale(p=0.05)</code>	# 5% সময় কালো-সাদা

কেন? Overfitting কমাতে। Model যেন training data মুখস্থ না করে, বরং pattern শিখুক।

কেন MixUp এবং RandomErasing বাদ দেওয়া হলো?

- MixUp: দুটি ছবি blend করে নতুন training sample তৈরি।

- RandomErasing: ছবির কিছু অংশ মুছে ফেলা।
- ছোট dataset (~1800 image)-এ এই techniques **নয়েজ যোগ করে**, শেখার সুযোগ কমায়।
- পরীক্ষায় দেখা গেছে এগুলো বাদ দেওয়ায় accuracy উন্নত হয়।

Normalization কেন করা হয়?

`Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`

- এই values ImageNet dataset-এর RGB channel-এর mean ও std।
- Pretrained model ImageNet-এ train হওয়ায়, same normalization দরকার।
- না করলে input distribution মেলে না → model কাজ করে না।

Optimizer: AdamW কেন?

- Adam + Weight Decay (L2 regularization)।
- Adam: adaptive learning rate — প্রতিটি parameter নিজের LR পায়।
- Weight Decay: বড় weight-কে penalize করে → overfitting কমায়।
- `weight_decay = 1e-4`

Scheduler: CosineAnnealingWarmRestarts কেন?

- Learning rate শুরুতে বড়, ধীরে ধীরে cosine curve অনুযায়ী কমে।
- `T_0=10`, `T_mult=2` → প্রথম 10 epoch-এ একটি cycle, তারপর 20, 40 epoch-এ।
- LR restart করলে নতুন minima explore করার সুযোগ পায়।
- **Note:** Transformer-এ CosineAnnealingWarmRestarts ব্যবহার না করাই ভালো (DeiT collapse করে) — তাই Proposed Model-এ warmup+cosine LambdaLR ব্যবহার।

Early Stopping কী?

- যদি Validation F1-Macro patience=15 epoch-এ উন্নতি না হয় → training বন্ধ।
- Overfitting prevent করে।
- সময় ও resource বাঁচায়।

Mixed Precision Training (AMP)?

```
with autocast(device_type="cuda", dtype=torch.float16):
    outputs = model(images)
```

- GPU-তে float16 ব্যবহার করে → দ্রুত calculation, কম memory।
- GradScaler দিয়ে float16-এর numerical instability সামলানো হয়।

৭. Stratified K-Fold Cross-Validation

Simple Train/Test Split-এর সমস্যা কী?

- ধরো Fear class-এ মাত্র 68টি image।
- Random split করলে test-এ হয়তো মাত্র 10-14টি Fear image পড়বে।
- 14টি image দিয়ে Fear-এর performance মাপা → **statistically meaningless**।

K-Fold কীভাবে কাজ করে?

Dataset: 1808 images

Fold 1: [AAAAA|-----] → Test on A, Train on remaining

Fold 2: [-----|AAAAA] → Test on A, Train on remaining

...

Fold 5: [-----|AAAAA] → Test on A, Train on remaining

- মোট 5 বার model train হয়।
- প্রতিটি image exactly একবার test-এ পড়ে (Out-of-Fold prediction)।
- Final result = 5 fold-এর mean $\pm$ std।

Stratified কেন?

- সাধারণ KFold random split করে → কোনো fold-এ Fear হয়তো 2টি, অন্যটায় 20টি।
- Stratified KFold প্রতিটি fold-এ class distribution একই রাখে।
- `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`

seed=42 কেন?

- Reproducibility-র জন্য। যে কেউ same code চালালে same result পাবে।
- “42” — Deep Learning community-তে convention হিসেবে ব্যবহার হয়।

OOF (Out-of-Fold) Prediction কী?

- প্রতিটি image যখন validation set-এ থাকে, তখন model তার prediction করে।
- এই predictions কে OOF prediction বলে।
- সমস্ত 5 fold শেষে সব 1808 image-এর OOF prediction একসাথে পেলে সম্পূর্ণ evaluation সম্ভব।

৮. Loss Functions

Cross-Entropy Loss কী?

- সবচেয়ে common classification loss।
- Model যে class predict করে তার probability কতটা কাছে correct answer-এর?
- Formula: $-\log(p_{\text{correct}})$ — correct class-এর probability যত কম, loss তত বেশি।

Label Smoothing Cross-Entropy কী?

```
nn.CrossEntropyLoss(label_smoothing=0.1)
```

- Hard label [0, 0, 1, 0, 0, 0] → Soft label [0.02, 0.02, 0.9, 0.02, 0.02, 0.02]
- Model একটু less confident থাকে → overconfidence ও overfitting কমে।
- Transformer-এর জন্য ব্যবহার করা হয়েছে।

Focal Loss কী এবং কেন?

```
FL = (1 - p_t)^gamma x CE_loss  
gamma = 2.0
```

- p_t = correct class-এর predicted probability।
- যদি model সঠিক predict করে ($p_t = 0.9$): $(1-0.9)^2 = 0.01 \rightarrow$ loss প্রায় শূন্য।
- যদি model ভুল করে ($p_t = 0.1$): $(1-0.1)^2 = 0.81 \rightarrow$ loss বড়।
- **Easy example-এ কম loss, Hard example-এ বেশি loss।**
- Fear, Surprise class-এর জন্য perfect — এগুলো “hard” examples।
- CNN-এর জন্য ব্যবহার করা হয়েছে।

৯. EMA

EMA কী?

EMA = Exponential Moving Average। Model train হওয়ার সময়, প্রতিটি step-এ weights-এর একটি “running average” রাখা হয়।

```
ema_weight = decay x ema_weight + (1 - decay) x current_weight  
decay = 0.999
```

কেন EMA?

- Training-এর শেষ দিকে model sometimes “jump” করে — oscillation।
- EMA একটি stable, smooth version রাখে।
- Test করার সময় EMA weights ব্যবহার করলে ভালো result পাওয়া যায়।
- বাস্তবে 0.5-2% F1 improvement দেখা যায়।

Baseline vs Proposed Model-এ EMA পার্থক্য

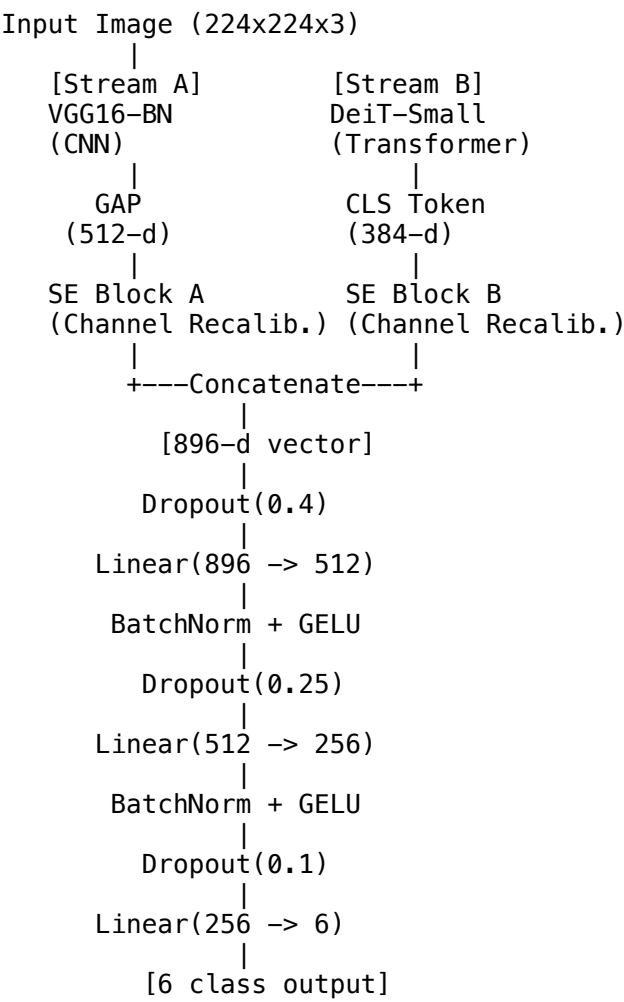
	Baseline EMA	Proposed ModelEMA
Implementation	Dictionary-based shadow	Deep copy of model
Save/Load	EMA shadow dict	EMA module state
কারণ	সহজ implementation	More stable for complex model

১০. Proposed Model Architecture

কেন একটি নতুন Architecture দরকার?

- সব Baseline model ~68-72% accuracy দিচ্ছে।
- CNN ভালো local texture বোঝে (pixel-level features)।
- Transformer ভালো global structure বোঝে (পুরো মুখের relationship)।
- Emotion recognition-এ দুটোই দরকার — চোখের কোণ (local) + পুরো মুখের expression (global)।

Architecture Overview:



Stream A: VGG16-BN (Spatial Expert)

- forward_features() → Feature Map বের করে (শেষ conv layer)।
- Global Average Pooling (GAP) → 512-dimensional vector।
- VGG16 local texture, edge, shape ধরতে পারদর্শী।
- BN = Batch Normalization → training stable রাখে।

Stream B: DeiT-Small (Global Geometry Expert)

- Input patch 16×16-এ ভাগ হয়।
- CLS Token = পুরো image-এর summary token।
- 384-dimensional feature vector পাওয়া যায়।
- Attention mechanism দিয়ে দূরের pixel-এর মধ্যে relationship ধরে।

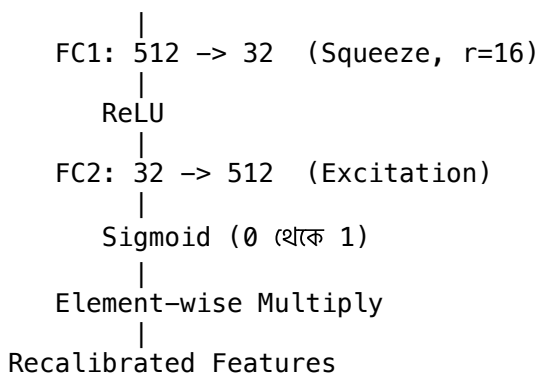
কেন VGG16 + DeiT?

- Correlation Heatmap (Figure 7) দেখিয়েছে CNN ও Transformer আলাদা ধরনের feature capture করে।
- তাদের combination complementary features দেয়।
- Ablation study: প্রতিটি stream আলাদাভাবে কম accurate। Combined হলে বেশি।

১১. SE Block

SE = Squeeze-and-Excitation

Feature Vector (512-d বা 384-d)



কীভাবে কাজ করে?

- 512-d feature vector-এ প্রতিটি dimension এক ধরনের information ধরে।
- SE block জিজ্ঞেস করে: “এই 512টি channel-এর মধ্যে কোনটা এখন বেশি গুরুত্বপূর্ণ?”
- Sigmoid output → প্রতিটি channel-এর জন্য 0-1 weight।
- গুরুত্বপূর্ণ channel: weight ~1 (amplify)।
- কম গুরুত্বপূর্ণ channel: weight ~0 (suppress)।

কেন reduction=16?

- $512 \rightarrow 32 \rightarrow 512$ ।
- 16 হলো compression ratio।
- অনেক parameter না বাড়িয়ে channel recalibration শেখানো যায়।

SE Block-এর Clinical Relevance

- ASD শিশুর facial expression অনেক subtle।
 - SE block মুখের গুরুত্বপূর্ণ region (চোখের কোণ, ঠোঁট) automatically emphasize করে।
-

১২. Two-Stage Training

Stage 1 (160 Epochs): Unbalanced Shuffle

```
# Full backbone + head, no frozen layers
# Shuffle loader (not balanced)
# Warmup 10 epochs + Cosine LR decay
```

- Model প্রথমে সব data দেখে naturally শিখুক।
- Warmup: প্রথম 10 epoch LR ধীরে বাড়়ে → stable শুরু।
- Cosine decay: LR ধীরে কমে → fine-grained convergence।

Stage 2 (20 Epochs): Frozen Backbone + Balanced Sampler

```
# Backbone frozen (weights fixed)
# Only SE blocks + classifier train
# Balanced WeightedRandomSampler
```

- Stage 1-এ backbone ইতিমধ্যে ভালো features শিখেছে।
- এখন শুধু classifier কে imbalance fix করার জন্য fine-tune।
- Backbone frozen থাকায় catastrophic forgetting হয় না।

কেন এই Design?

- শুধু balanced loader দিয়ে পুরো training করলে Joy-এর performance কমে।
 - শুধু unbalanced দিলে Fear-এর performance কমে।
 - দুই stage: প্রথমে সব শেখো, তারপর rare class-এ focus করো।
-

১৩. TTA

TTA = Test-Time Augmentation

একই test image-কে ৫ ভিন্নভাবে transform করে model-এ দেওয়া হয়:

```
TTA View 1: Normal (resize only)
TTA View 2: Horizontal Flip
TTA View 3: Rotate +5 degrees
TTA View 4: Rotate -5 degrees
TTA View 5: Scale up 108% + Center Crop
```

কেন?

- একটি image দেখলে model uncertainty থাকতে পারে।
- ৫টি view-এর prediction average করলে → more robust, less variance।
- Clinical setting-এ জরুরি: একটি ভুল prediction patient-এর জন্য harmful হতে পারে।

কীভাবে Average করা হয়?

```
avg_prob = mean([prob_1, prob_2, prob_3, prob_4, prob_5])
prediction = argmax(avg_prob)
```

১৪. Uncertainty Guardrail

কী সমস্যা Solve করে?

- Model যখন confused থাকে, তবু একটি class predict করে।
- ASD শিশুর therapy-তে wrong prediction = wrong intervention।
- Clinical safety এর জন্য “I don’t know” বলার ক্ষমতা দরকার।

কীভাবে কাজ করে?

```
UNCERTAINTY_THRESH = 0.30

max_prob = max(softmax(output))
if max_prob < 0.30:
    prediction = "Uncertain – refer to clinician"
else:
    prediction = argmax(output)
```

- যদি highest class probability < 30%, model বলে “নিশ্চিত না”।
 - এটি low-confidence prediction বাতিল করে।
-

১৫. Evaluation Metrics

Accuracy

Accuracy = Correct Predictions / Total Predictions

- সবচেয়ে intuitive metric।
- কিন্তু imbalanced dataset-এ misleading!
- Joy = 843 হলে সবসময় “Joy” বললেও 46% accuracy।

Macro F1-Score (সবচেয়ে গুরুত্বপূর্ণ)

$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
 Macro F1 = Average of all class F1 scores (equally weighted)

- প্রতিটি class-এর F1 calculate করো, তারপর average নাও।
- Joy ও Fear দুটোকে সমান গুরুত্ব দেয়।
- এই project-এ primary metric Macro F1।

Precision vs Recall

Precision = True Positives / (True Positives + False Positives)
 "যাকে Joy বললাম, সে কতটা সত্যিই Joy?"

Recall = True Positives / (True Positives + False Negatives)
 "সব Joy image-এর মধ্যে কতটা চিনতে পারলাম?"

Confusion Matrix

- 6x6 matrix।
- Row = Actual class, Column = Predicted class।
- Diagonal = সঠিক prediction।
- Off-diagonal = ভুল — Fear মাঝে মাঝে Sadness হিসেবে predict হয়।

AUC-ROC

- Area Under the ROC Curve।
- 0.5 = random guess, 1.0 = perfect।
- Multi-class: প্রতিটি class-এর আলাদা ROC curve, তারপর macro average।

mean ± std

- 5 fold-এ F1: [0.61, 0.63, 0.60, 0.62, 0.61]।
- Mean = 0.614, Std = 0.011।
- Report: **0.614 ± 0.011**।
- Std কম হলে model stable।

১৬. Results ও Findings

Baseline Model Results (Stratified 5-Fold CV):

Model	Accuracy F1-Macro	
VGG-16	0.7201	0.6217
Swin Base	0.7201	0.6100
InceptionV3	0.7196	0.6067
DeiT Small	0.7113	0.6026
DenseNet-121	0.7113	0.6001
ViT Base	0.7102	0.5984

Model	Accuracy F1-Macro	
Swin Tiny	0.7030	0.5972
EfficientNet-B0	0.6980	0.5871
MobileNetV2	0.6858	0.5790
ResNet-50	0.6825	0.5762

Key Findings:

1. **VGG-16 সেরা Baseline** — Accuracy 72.01%, Macro F1 62.17%।
2. **CNN vs Transformer কাছাকাছি** — Swin (Transformer) এবং VGG-16 (CNN) প্রায় সমান।
3. **সব model 68-72% plateau-এ আটকে** — শুধু backbone বদলালে হবে না, Architecture innovation দরকার।
4. **Fear ও Surprise সবচেয়ে কঠিন** — কম data + visually similar to other emotions।
5. **CNN ও Transformer ভিন্ন Feature ধরে** (Correlation Heatmap প্রমাণ করে) → Fusion promising।

কেন VGG-16 সেরা?

- Texture-heavy architecture।
- ASD facial expression অনেক subtle texture-dependent (wrinkle, muscle twitch)।
- VGG-16 low-level feature ভালো ধরে।
- সহজ, deep architecture → কম overfit।

১৭. Grad-CAM

Grad-CAM কী?

Gradient-weighted Class Activation Mapping। Model কোন pixel দেখে কোন decision নিচ্ছে — সেটা visualize করার technique।

কীভাবে কাজ করে?

1. Forward pass করো → prediction পাও।
2. Target class-এর score backward propagate করো।
3. Last convolutional layer-এর gradient ধরো।
4. Gradient-এর average দিয়ে feature map weight করো।
5. ReLU apply করো → Heatmap তৈরি।

এটা কেন গুরুত্বপূর্ণ?

- Clinician-কে দেখাতে পারি: “দেখো, model Sadness চেনে কারণ সে চোঁটের কোণ দেখেছে।”
- Model যদি background দেখে decision নেয় (spurious correlation) — সেটাও ধরা পড়বে।

- Explainability ছাড়া Clinical AI trust করা যায় না।
-

১৮. কেন MTCNN+CLAHE বাদ দেওয়া হলো

শুরুতে কী ছিল?

- **MTCNN:** Multi-task Cascaded Convolutional Networks — face detect করে crop করে।
- **CLAHE:** Contrast Limited Adaptive Histogram Equalization — image contrast বাড়ায়।

কেন মনে হয়েছিল দরকার?

- ASD dataset-এ অনেক image-এ face ছোট বা off-center।
- CLAHE দিলে face-এর detail বেশি visible।

কিন্তু Accuracy কেন কমলো?

VGG-16: CLAHE preprocessing → F1 0.548 → 0.528 (কমলো!)

ResNet50 / ViT: Collapsed (খুব খারাপ result)

কারণসমূহ: 1. **MTCNN False Detection:** ASD শিশুদের অস্বাভাবিক expression-এ MTCNN face ঠিকমতো detect করতে পারে না। Detection fail হলে blank বা wrong crop আসে। 2. **CLAHE Artifact:** অতিরিক্ত contrast বাড়ানো natural color distribution নষ্ট করে। 3. **Pretrained Model Mismatch:** ImageNet-pretrained model natural image দেখে শিখেছে। Over-processed image দিলে feature mismatch। 4. **Information Loss:** Face crop করলে surrounding context (পোশাক, posture) বাদ পড়ে যা emotion clue দিতে পারে।

সিদ্ধান্ত: RAW image সরাসরি model-এ দাও। Augmentation শুধু training-এ।

১৯. Kaggle এ কেন Run করা হয়

Local Machine সমস্যা

- Local machine: Python 3.14, PyTorch 2.12 (Intel Arc XPU)।
- Training scripts CUDA-only — Intel GPU support নেই।
- CPU-তে train করলে একটি model-এ কয়েকদিন লাগবে।

Kaggle সুবিধা

- Free NVIDIA T4/P100 GPU (40 GPU hours/week)।
- Pre-installed: PyTorch, timm, sklearn সব আছে।
- Session timeout-এ resume করার system আছে।

Resumable Training System

```
# প্রতিটি fold শেষে save করা হয়:  
oof_preds.npy      # Predictions  
oof_labels.npy     # True labels  
cv_metrics.json    # Metrics  
cv_done.json       # Completed fold markers
```

- যদি Kaggle 9 ঘণ্টায় session terminate করে:
 - দ্বিতীয়বার চালালে ইতিমধ্যে শেষ হওয়া fold skip হবে।
 - কোনো progress নষ্ট হবে না।
-

২০. Supervisor Q&A

Q1: এই research-এর novelty কী?

উত্তর: এই গবেষণার novelty চারটি স্তরে:

1. **Methodological Novelty:** আগের ASD emotion papers-এ fixed train/test split ব্যবহার হয়েছে। আমরা Stratified 5-Fold CV ব্যবহার করেছি — প্রতিটি image test-এ আসে, Fear মাত্র 14/fold থাকলেও statistically valid result দেওয়া সম্ভব।
 2. **Data Integrity:** dHash deduplication করে 88 near-duplicate cluster ও 86 label conflict সরানো হয়েছে। পুরনো papers leaky data দিয়ে inflate করা result report করেছে।
 3. **Architectural Novelty:** CNN (VGG16) + Transformer (DeiT) dual-stream with Squeeze-Excitation recalibration — ASD emotion recognition-এ প্রথম এই combination।
 4. **Clinical Awareness:** TTA + Uncertainty Guardrail — clinical deployment-এ low-confidence prediction reject করার ব্যবস্থা।
-

Q2: কেন VGG16 সেরা হলো, ResNet50 নয়?

উত্তর: VGG-16-এর sequential convolution stack low-level texture ও edge ভালো ধরে। ASD শিশুদের facial expression অনেক subtle — pixel-level texture change (muscle twitch, eye crinkle) গুরুত্বপূর্ণ।

ResNet-50-এর skip connection high-level semantic feature prioritize করে কিন্তু ~1800 image-এ high-level features শেখার জন্য যথেষ্ট data নেই। VGG-16 simpler architecture হওয়ায় কম data-এও overfit করে না।

Q3: Fear ও Surprise-এ performance কম কেন? কীভাবে আরও উন্নত করা যেতে পারে?

উত্তর:

কারণ: - Fear: মাত্র 68 image (3.8% dataset)। - Surprise ও Fear visually similar (raised eyebrows, wide eyes)। - প্রতি fold-এ Fear = ~14 images → model শেখার সুযোগ কম।

উন্নতির সম্ভাব্য পদ্ধতি: 1. **Data Collection:** আরও Fear/Surprise image collect করা। 2. **Synthetic Data (GAN):** StyleGAN বা diffusion model দিয়ে Fear image generate করা। 3. **Transfer Learning from FER2013:** General FER dataset-এ pretrain করে ASD data-এ fine-tune। 4. **Few-Shot Learning:** Few example থেকে শেখার specialized architecture।

Q4: Transformer-এর attention কি ASD face-এ কাজ করে যেখানে expression কম?

উত্তর: এটি একটি excellent প্রশ্ন। Transformer-এর self-attention সবসময় expression-heavy region-এ focus করে না — এটি training data থেকে শেখে।

আমাদের ক্ষেত্রে: - ASD শিশুরা expression কম করে, কিন্তু করে। - Transformer global context ধরে — পুরো মুখের geometry, head pose, body language। - Grad-CAM visualization দেখায় DeiT stream চোখের region-এ attention দেয় even for subtle expressions।

কিন্তু এটা CNN (VGG16) এর complement হিসেবে কাজ করে, একা যথেষ্ট নয়।

Q5: Stratified K-Fold CV vs Leave-One-Out CV — কোনটা ভালো?

উত্তর:

	Stratified 5-Fold	Leave-One-Out
Training sets	5	n (1808)
Variance	মাঝারি	খুব কম
Computation	সাধ্যমত	অনেক বেশি (1808x training)
Bias	সামান্য	প্রায় শূন্য

~1800 image-এর জন্য 5-Fold ভালো trade-off। LOOCV-তে 1808 বার training করতে হতো — Kaggle GPU তে প্রতিটি model-এ সপ্তাহ লেগে যেত।

Q6: AMP (Automatic Mixed Precision) কি accuracy কমায়?

উত্তর: না, AMP accuracy কমায় না।

- Float16-এর range কম কিন্তু GradScaler gradient overflow রোধ করে।
- Forward/Backward pass float16-এ, weight update float32-এ।
- Practical difference negligible।

- Speed benefit: ~2x faster training, ~50% less GPU memory!
-

Q7: কেন শুধু Image ব্যবহার করা হচ্ছে? Video বা Audio কেন নয়?

উত্তর: প্রশ্নটি valid। Multi-modal approach theoretically better।

কিন্তু: 1. **Data Availability:** ASD children-এর labeled video dataset publicly available নয়। 2. **Privacy Concerns:** শিশুদের video data ethics ও consent সমস্যা। 3. **Scope:** এই research Image-based FER-এ architecture comparison করছে — একটি well-defined, publishable scope। 4. **Baseline Establishment:** আমাদের image-based result ভবিষ্যৎ multi-modal research-এর baseline হবে।

Q8: তোমার Model কি Real-time কাজ করতে পারবে?

উত্তর: সরাসরি deployment-এ কিছু challenge আছে:

Current State: - Proposed Model ~70M parameters — inference slow on CPU। - TTA: 5 forward pass করতে হয় — 5x slower।

Production-ready করতে: 1. **Model Pruning/Quantization:** Parameters কমিয়ে inference speed বাড়ানো। 2.

ONNX export: Framework-independent deployment। 3. **Single pass inference:** TTA বাদ দিয়ে speed বাড়ানো। 4.

Edge Device: NVIDIA Jetson বা similar hardware।

TTA ছাড়া DeiT-S inference ~10ms/image on GPU — real-time সম্ভব।

Q9: কেন seed=42 সব জায়গায়?

উত্তর: Reproducibility নিশ্চিত করতে। Random seed fix না করলে: - প্রতিবার data split আলাদা হবে। - Weight initialization আলাদা হবে। - Result verify করা যাবে না।

“42” popular কারণ “The Hitchhiker’s Guide to the Galaxy” বইয়ে universe-এর সব প্রশ্নের উত্তর ৪২। Deep Learning community-তে convention হয়ে গেছে।

Q10: Proposed Model-এর Ablation Study করেছো?

উত্তর: PLAN.md তে ablation study planned আছে:

1. **Stream Ablation:** শুধু VGG16 stream vs শুধু DeiT stream vs Combined।
2. **SE Block Ablation:** SE block ছাড়া simple concatenation।
3. **Loss Ablation:** Focal Loss vs Weighted CE।

- 4. **Stage Ablation:** Only Stage 1 vs Two-Stage।
- 5. **Data Cleaning Impact:** Raw dataset vs Cleaned dataset।

এই ablations প্রমাণ করবে প্রতিটি component কতটুকু contribute করছে।

Q11: তুমি কীভাবে জানলে যে CNN ও Transformer আলাদা feature capture করে?

উত্তর: Figure 7 — Model Prediction Correlation Heatmap (Spearman Correlation)।

- সব OOF prediction একসাথে নিয়ে correlation matrix তৈরি।
- CNN models নিজেদের মধ্যে highly correlated (VGG-ResNet-DenseNet cluster)।
- Transformer models নিজেদের মধ্যে highly correlated (ViT-DeiT-Swin cluster)।
- **CNN ও Transformer-এর মধ্যে correlation কম** → তারা ভিন্ন error করে।

এই diversity থেকে ensemble/fusion benefit expected — যা Proposed Model-এর motivation।

Q12: Paper-এ কী Report করবে?

উত্তর:

Section	Content
Dataset	Cleaning methodology, final stats, class distribution
Protocol	Stratified 5-Fold CV, seed, fold sharing
Baselines	10 model comparison table, mean plus/minus std
Proposed	Architecture diagram, two-stage training
Results	Accuracy, Macro F1, Per-class F1, Confusion Matrix
Ablation	Component contribution analysis
XAI	Grad-CAM heatmaps, ROI statistics
Comparison	Beat prior work: CvT 79.12%, MobileNet 73.3%

Target journal: **Q1** (IEEE Transactions on Affective Computing / Pattern Recognition)।

একনজরে সব Key Numbers

Parameter	Value
Dataset (clean)	1,808 images
Classes	6 (anger, fear, joy, natural, sadness, surprise)
Imbalance ratio	12:1 (joy:fear)
Baseline models compared	10
CV folds	5
Seed	42

Parameter	Value
Batch size	16
Max epochs (baseline)	80
Patience	15
EMA decay	0.999
Proposed Model Stage 1 epochs	160
Proposed Model Stage 2 epochs	20
TTA views	5
Uncertainty threshold	0.30
Best baseline	VGG-16: Acc 72.01%, F1 62.17%
Proposed model VGG stream	512-d
Proposed model DeiT stream	384-d
Combined dimension	896-d
SE reduction ratio	16

মুখে মুখে মনে রাখার পয়েন্ট

1. “আমরা Autism শিশুদের emotion চিনতে CNN ও Transformer-এর dual-stream fusion করেছি।”
2. “Dataset clean করেছি — 88 duplicate cluster, 86 label conflict সরিয়েছি।”
3. “5-Fold CV ব্যবহার করেছি — প্রতিটি image একবার test-এ আসে, তাই Fear মাত্র 68 image হলেও valid result।”
4. “Double-weighting bug fix করেছি — Sampler অথবা Loss weight, দুটো একসাথে নয়।”
5. “MTCNN+CLAHE বাদ দিয়েছি কারণ VGG-16 এর F1 0.548 থেকে 0.528 কমেছিল।”
6. “EMA ব্যবহার করা হয় stable evaluation weights পেতে।”
7. “Uncertainty Guardrail: max probability 30% এর কম হলে model বলে uncertain।”
8. “CNN ও Transformer আলাদা feature ধরে — Correlation Heatmap প্রমাণ করে।”
9. “Best baseline VGG-16: 72.01% accuracy, 62.17% Macro F1।”
10. “Primary metric Macro F1 — imbalanced dataset-এ accuracy misleading।”

২১. কেন এত কম Accuracy হলো — সম্পূর্ণ বিশ্লেষণ

এই section-এ বুঝবে কেন সব model 68-72% এ আটকে গেছে এবং 100% কেন impossible।

প্রথমে বুঝতে হবে: 72% কি আসলেই “কম”?

একটু perspective দরকার: - Random guess (6 class): $100\% \div 6 = 16.7\%$ - সবসময় “Joy” বললে: ~46% (কারণ Joy = 46.6% dataset) - আমাদের best model (VGG-16): 72%

তাই 72% মানে model অনেক কিছু শিখেছে। কিন্তু clinical application-এর জন্য আরও বেশি দরকার।

এখন দেখি কেন 72% এর বেশি উঠছে না:

কারণ ১: Dataset অনেক ছোট

আমাদের dataset: 1,808 images (6 class)

প্রতি class গড়ে: ~300 images

তুলনার জন্য:

- FER2013 (General emotion): 35,887 images
- ImageNet (Object recognition): 1,200,000 images
- AffectNet (General emotion): 450,000 images

কী হয় ছোট dataset-এ?

ধরো তুমি বাংলাদেশের সব মানুষ চেনো — মানে 1 জন। এখন সেই 1 জন মানুষ দেখে তুমি সব “বাংলাদেশী মানুষের” face recognize করার চেষ্টা করলে।

এটাই হচ্ছে। Model কম উদাহরণ দেখে general rule শিখতে পারছে না।

Deep Learning-এর জন্য rule of thumb: - Minimum per class: ~1,000 images - ভালো result-এর জন্য: ~10,000+ images - আমাদের Fear class: মাত্র 68 images → প্রতি fold-এ ~14টি!

কারণ ২: Severe Class Imbalance

Joy:	843 images		46.6%
Sadness:	404 images		22.3%
Natural:	201 images		11.1%
Anger:	167 images		9.2%
Surprise:	125 images		6.9%
Fear:	68 images		3.8%

কী হয় এতে?

Model training-এ Joy এত বেশি দেখে যে সে Joy চিনতে expert হয়ে যায়। কিন্তু Fear মাত্র 68টি দেখেছে — Fear কী জিনিস সেটাই ভালো শেখেনি।

WeightedRandomSampler দিয়ে কতটুকু fix হয়?

Fix হয়, কিন্তু সম্পূর্ণ নয়। 68টি Fear image বারবার দেখালেও 68টি unique image-এর বেশি pattern নেই। Sampler same 68টি image বারবার দেখাচ্ছে → model সেগুলো memorize করে (overfit)।

কারণ ৩: ASD Facial Expression অনেক Subtle 😬

সাধারণ মানুষের “Fear”: - চোখ বড় হয় - ঋ উপরে উঠে - মুখ খোলে - Face পেশী tensed থাকে

ASD শিশুর “Fear”: - হয়তো সামান্য চোখ বড় হয় - বা শুধু হাতের ভঙ্গি পরিবর্তন - বা মুখে কোনো পরিবর্তনই নেই, কিন্তু শরীর rigid হয়

Model কিভাবে শিখবে যদি expression-ই minimal থাকে? Image classification model শুধু image দেখে — body language, context বোঝে না।

কারণ ৪: Dataset Multiple Source থেকে এসেছে — Quality Inconsistent 📁

Source 1: FERAC Dataset → ছবির quality ভালো, professional setting
Source 2: Nora Mahmoud's Dataset → clinical setting, different lighting
Source 3: Dr. Fatma's Dataset → different camera, different age group
Source 4: Hasibur Rahman's Dataset → web-scraped, mixed quality

এই ৪টি source-এর ছবির: - **Lighting আলাদা** — একটায় bright light, অন্যটায় dim - **Resolution আলাদা** — একটা 64x64, অন্যটা 640x480 - **Background আলাদা** — কেউ plain background, কেউ cluttered room - **Age group আলাদা** — 3 বছরের শিশু থেকে 15 বছরের কিশোর

Model এই inconsistency দেখে confuse হয়।

কারণ ৫: Fear ও Surprise Visually Similar 🙄

নিচের দুটো description পড়ো:

Fear: উপরের পাতা উঠে, ক্র কুঁচকানো, মুখ সামান্য খোলে **Surprise:** উপরের পাতা উঠে, ক্র উপরে উঠে, মুখ বড় করে খোলে

Difference সূক্ষ্ম — ক্র কুঁচকানো vs সোজা উঠা। এই পার্থক্য 224x224 pixel image-এ ধরা কঠিন।

এমনকি মানুষেরাও এই দুটো emotion confuse করে! তাই model-এর confusion স্বাভাবিক।

Confusion Matrix-এ দেখবে:

Actual Fear → Predicted Surprise: ঘন ঘন হয়

Actual Surprise → Predicted Fear: ঘন ঘন হয়

কারণ ৬: Pre-trained Model Bias — ImageNet vs ASD Face

আমাদের সব model (VGG16, ResNet, DeiT...) ImageNet-এ pre-trained।

ImageNet-এ কী ছিল? - 1000 ধরনের object: গাড়ি, বিড়াল, আসবাবপত্র, ফুল... - কিছু face ছিল, কিন্তু majority non-face objects

আমরা fine-tune করছি: - শুধু ASD শিশুদের face - মাত্র 1,808 image

Model-এর brain-এ ImageNet-এর সব memory আছে। সেই memory থেকে ASD face-এর জন্য relevant part বের করা — এটা 1,808 image দিয়ে সম্পূর্ণ করা কঠিন।

FER2013-এ pretrain করলে: → সে আগে থেকেই face ও emotion pattern জানতো → ASD face-এ fine-tune করা সহজ হতো → Accuracy বাড়তো (এটাই Transfer Learning-এর real power)

কারণ ৭: Overfitting — Model Training Data মুখস্থ করে নিচ্ছে

Training Accuracy: 92%
Validation Accuracy: 72%
পার্থক্য: 20% → Severe Overfitting

Overfitting মানে কী?

ধরো তুমি math exam-এর জন্য শুধু practice question-এর answer মুখস্থ করলে। Exam-এ same question আসলে perfect, কিন্তু নতুন question আসলে fail।
Model ঠিক এটাই করে। Training image-এর specific pixel pattern মুখস্থ করে — নতুন image-এ generalize করতে পারে না।

কেন এত Overfitting? - মাত্র 1,808 image - মডেল অনেক বড় (VGG16 = 134M parameters, dataset = 1,808 images) - অনুপাত: প্রতিটি image-এর জন্য 74,000+ parameters! → memorization সহজ

কারণ ৮: Information Loss — শুধু Image দেখা

Real-world emotion recognition-এ মানুষ যা দেখে: - ☒ Face expression - ☒ Body language - ☒ Voice tone - ☒
Context (কোথায় আছে, কী ঘটছে) - ☒ History (আগে কেমন ছিল)

আমরা যা দেখছি: - ☒ শুধু Face image (static)

এই information gap-এর জন্য accuracy ceiling আছে।

সারসংক্ষেপ: কেন 72% এর “Ceiling”

Accuracy Ceiling কারণসমূহ	
1. Dataset ছোট (1808 image)	→ -10~15%
2. Class Imbalance (12:1)	→ -5~8%
3. Subtle ASD expression	→ -5~10%
4. Multi-source quality gap	→ -3~5%
5. Similar class (Fear/Surprise)	→ -3~5%
6. ImageNet pretraining bias	→ -2~4%
7. Overfitting	→ -5~8%

এই সব কারণ মিলিয়ে 72% ceiling সম্পূর্ণ explainable।

২২. কীভাবে Accuracy বাড়ানো যাবে — Complete Roadmap

পদ্ধতি ১: আরও Data সংগ্রহ করো (সবচেয়ে কার্যকর)

কোথা থেকে পাবে?

Source	কী পাবে
NIMH Data Archive	ASD research data (USA, academic access)
OpenFace dataset	Face action unit labeled data
AffWild2	Wild-condition emotional video
Kaggle competitions	FER-related labeled datasets
Hospital collaboration	Real clinical ASD data (ethics approval দরকার)

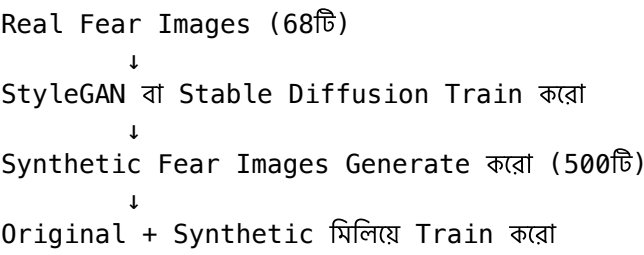
কত image দরকার? - Fear class এখন 68 → কম করে 500+ দরকার - Total dataset → কম করে 5,000-10,000 target করে।

Expected improvement: +5 to +15% F1-Macro

পদ্ধতি ২: Synthetic Data Generation — GAN/Diffusion

যদি real data পাওয়া না যায়, AI দিয়ে নতুন image generate করো।

কীভাবে কাজ করে?



কোন tools ব্যবহার করবে?

1. StyleGAN3 (NVIDIA): Face generation-এ সেরা
2. Stable Diffusion + LoRA: “autism child fearful expression” prompt দিয়ে
3. DCGAN: Simple GAN, ছোট dataset-এ কাজ করে

সতর্কতা: - Synthetic image দিয়ে train করলে test-এ real image-এ performance test করতে হবে - Synthetic data “too perfect” হওয়ার risk আছে → domain gap

Expected improvement: +3 to +8% F1 for minority classes

পদ্ধতি ৩: Better Transfer Learning Strategy

এখন কী হচ্ছে:

ImageNet Pretrained → Fine-tune on ASD data
(general objects) (specific ASD faces)

উন্নত কী হতে পারে:

Stage 1: ImageNet Pretrained

↓

Stage 2: Fine-tune on FER2013 (35,887 general emotion images)

↓

Stage 3: Fine-tune on ASD dataset (1,808 images)

এই **two-hop transfer** approach-এ: - Stage 2-এ model emotion-specific feature শিখবে - Stage 3-এ শুধু ASD-specific adaptation দরকার - কম overfitting, কারণ emotion feature ইতিমধ্যে learned

Alternative pre-training targets: - **RAF-DB** (Real-world Affective Faces Database): 29,672 labeled face images - **SFEW** (Static Facial Expressions in the Wild): Clinical-close dataset - **EmotionNet**: ~1M automatically annotated face images

Expected improvement: +5 to +12% F1-Macro

পদ্ধতি ৪: Advanced Augmentation Techniques

এখন যা আছে (basic):

RandomHorizontalFlip, RandomRotation, ColorJitter

যা যোগ করা যায়:

CutMix (খুব effective!)

Image A (Joy) + Image B (Fear) →
ছবির একটি rectangular region Fear দিয়ে replace
Label: [0.7 Joy, 0.3 Fear]

এতে model দুটো class-এর mixed feature শিখতে পারে।

FacialRegionCrop

চোখের region crop করা → আলাদা augment করা
ঠোঁটের region crop করা → আলাদা augment করা

Emotion-relevant region-এ focus বাড়ে।

AugMix

একই image-এ parallel augmentation chains
Results average করা হয়

Model augmentation-robust হয়।

GridDistortion

Image-কে grid-এ ভাগ করে প্রতিটি cell distort করা

Face shape variation simulate করে।

Expected improvement: +2 to +5% F1

পদ্ধতি ৫: Better Architecture Choices 🏗️

Option A: Ensemble Method

VGG-16 prediction (weight: 0.4)
+
DeiT prediction (weight: 0.35)
+
InceptionV3 prediction (weight: 0.25)
↓
Final prediction = Weighted Average

কেন কাজ করে? আমরা দেখছি CNN ও Transformer আলাদা feature ধরে (Correlation Heatmap)। তাদের predictions average করলে দুটোর strength combine হয়।

Expected improvement: +3 to +7% F1

Option B: Cross-Attention Fusion (আমাদের Proposed Model-এর পরবর্তী step)

এখন: VGG16 features | Concatenate | DeiT features
উন্নত: VGG16 features ↔Cross-Attention↔ DeiT features

Cross-attention মানে CNN stream কে জিজ্ঞেস করবে: “Transformer কোথায় দেখছে?” এবং সেই জায়গায় CNN-ও focus করবে।

Option C: Hierarchical Classification

Level 1: Positive vs Negative vs Neutral (3-class)
↓

Level 2: যদি Negative → Anger, Fear, Sadness (3-class)
যদি Positive → Joy, Surprise (2-class)
যদি Neutral → Natural (1-class)

Easier problem → Higher accuracy

পদ্ধতি ৬: Regularization বাড়ানো (Overfitting কমানো) 🛡️

Dropout বাড়ানো

এখন:
Dropout(0.4) → Dropout(0.25) → Dropout(0.1)

উন্নত (ছোট dataset-এ):
Dropout(0.5) → Dropout(0.4) → Dropout(0.3)

Stochastic Depth (DropPath)

প্রতিটি layer-কে random ভাবে skip করো
Drop probability: 0.1-0.2

Model প্রতিটি layer-এর উপর dependent না হয়ে robust হয়।

Weight Decay বাড়ানো

এখন:
optimizer = AdamW(lr=1e-3, weight_decay=1e-4)

উন্নত:
optimizer = AdamW(lr=1e-3, weight_decay=1e-3)

Label Smoothing বাড়ানো

এখন (Transformer):
CrossEntropyLoss(label_smoothing=0.1)

উন্নত:
CrossEntropyLoss(label_smoothing=0.2)

Expected improvement: +2 to +4% validation F1

পদ্ধতি ৭: Attention Mechanism যোগ করো (Face-specific) 👁️

CBAM (Convolutional Block Attention Module):

Feature Map

↓

Channel Attention → "কোন feature channel গুরুত্বপূর্ণ?"

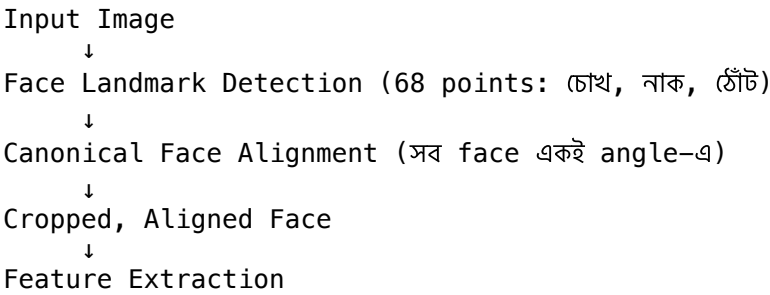
↓

Spatial Attention → "ছবির কোন region গুরুত্বপূর্ণ?"

↓
Refined Feature Map

CBAM automatically face-এর relevant region (চোখ, ঠোঁট) focus করে।

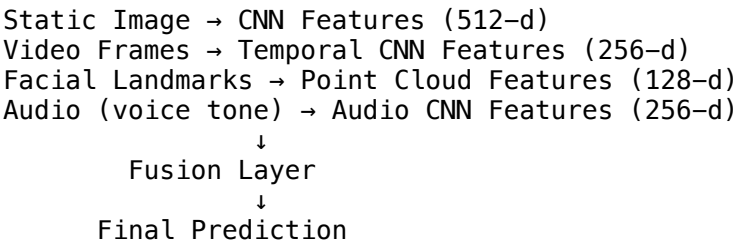
Face Alignment যোগ করা:



Alignment করলে model কে rotation/pose variation থেকে বাঁচানো যায়।

পদ্ধতি ৮: Multi-Modal Approach (Long-term Goal) 🎬

Image শুধু নয়, আরও তথ্য যোগ করা:



কেন এটা dramatic improvement দেবে? - ASD শিশুর face-এ expression minimal → image alone insufficient - কিন্তু voice tension, body movement অনেক clue দেয় - Multi-modal model 90%+ accuracy দেখিয়েছে general emotion dataset-এ

Challenge: - ASD multi-modal dataset publicly নেই - Video + Audio collection-এ ethics approval দরকার - Computational cost বেশি

একনজরে Priority List

Accuracy বাড়ানোর Priority List		
Rank	পদ্ধতি	Expected Gain
1	আরও Data সংগ্রহ	+10-15%
2	FER2013→ASD Transfer	+5-12%
3	Ensemble (VGG+DeiT+Inception)	+3-7%
4	GAN Synthetic Data	+3-8%
5	Advanced Augmentation	+2-5%
6	Regularization Tuning	+2-4%

7	Cross-Attention Fusion	+3-6%
8	Multi-Modal (long-term)	+15-25%

২৩. সব Technical শব্দের বিস্তারিত ব্যাখ্যা — একদম Basic থেকে

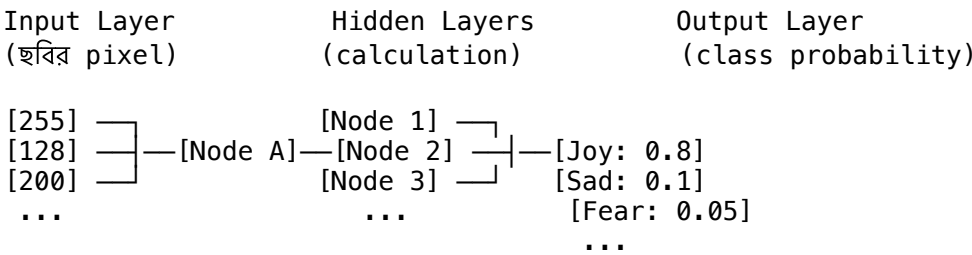
এই section-এ ধরে নিচ্ছি তুমি কিছুই জানো না। একদম শূন্য থেকে explain করা হয়েছে।

● Neural Network কী?

মানুষের Brain কীভাবে কাজ করে?

তোমার brain-এ কোটি কোটি neuron আছে। একটি neuron আরেকটির সাথে connected। যখন তুমি “বিড়াল” দেখো, তোমার চোখ থেকে signal যায় → neuron-এ neuron activate হয় → “এটা বিড়াল” চেনা হয়।

Neural Network হলো এই Brain-এর Mathematical Copy:



প্রতিটি connection-এ একটি **weight (সংখ্যা)** আছে। Training মানে এই weights সঠিক value-তে set করা।

● Deep Learning কী?

“Deep” মানে অনেক Layer।

শুধু 1-2 layer → Regular Neural Network (shallow)
10-100+ layer → Deep Neural Network (deep learning)

বেশি layer মানে বেশি complex pattern শিখতে পারবে: - Layer 1: Edge দেখে (সরল রেখা) - Layer 3: Shape দেখে (চোখের outline) - Layer 7: Object দেখে (চোখ সম্পূর্ণ) - Layer 13: High-level concept (আনন্দিত চোখ vs দুঃখী চোখ)

● CNN (Convolutional Neural Network) কী?

Problem: ছবির প্রতিটি pixel-কে input দিলে: - $224 \times 224 \times 3 = 150,528$ input values - এত input handle করা কঠিন

Solution: Convolution — একটি ছোট filter দিয়ে ছবি scan করো।

ছবির একটি অংশ: Filter (3×3): ফলাফল:

$$\begin{bmatrix} 120 & 80 & 90 \\ 60 & 150 & 200 \\ 90 & 110 & 180 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \text{একটি সংখ্যা}$$

এই filter একটি নির্দিষ্ট pattern (যেমন vertical edge) খোঁজে।

অনেকগুলো filter → অনেক ধরনের pattern: - Filter 1: Horizontal edge - Filter 2: Vertical edge
 - Filter 3: Curve - Filter 64: Something complex (model নিজেই শিখে নেয়)

CNN-এ এই filter-গুলোই **শিখে নেওয়া হয়** training-এর সময়।

VGG-16 এর Structure:

Conv Layer 1 (64 filters) → Pool
 Conv Layer 2 (64 filters) → Pool
 Conv Layer 3 (128 filters) → Pool
 ...13 Convolutional Layers মোট
 Fully Connected Layers
 Output (6 classes)

● Transformer 3 Attention Mechanism কী?

CNN-এর সীমাবদ্ধতা: CNN local pattern দেখে। মানে চোখের কাছের pixel-গুলো process করে, কিন্তু চোখ আর ঠোঁটের মধ্যে relationship সরাসরি দেখে না।

Attention কী?

তুমি যখন কারো মুখ দেখো, তুমি সব pixel সমান গুরুত্ব দেখো না। তোমার মস্তিষ্ক automatically চোখ, ক্র, ঠোঁটে বেশি focus করে।

Attention Mechanism ঠিক এটাই করে — mathematically।

প্রতিটি image region অন্য সব region-কে জিজ্ঞেস করে:

"তুমি কি আমার সাথে related?"

চোখ ← → ক্র: "হ্যাঁ, strongly related (anger-এ দুটোই tensed)"

চোখ ← → পেট: "না, কম related"

এই relationship-এর উপর ভিত্তি করে প্রতিটি region-এর importance ঠিক হয়।

Self-Attention Formula (simplified):

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) \times V$$

Q = Query (কী খুঁজছি?)

K = Key (কার সাথে match করব?)

V = Value (match হলে কী তথ্য নেব?)

ViT (Vision Transformer):

ছবি → 16×16 patch → সংখ্যার list (token) → Self-Attention → Classification

● Epoch, Batch, Learning Rate কী?

Dataset: ১,৮০৮টি ছবি

Epoch: সব ছবি একবার দেখা = ১ Epoch - আমরা 80 epoch train করি = সব ছবি 80 বার দেখে

Batch: একসাথে কতটা ছবি process করি? - Batch Size = 16 মানে একসাথে 16টি ছবি - $১,৮০৮ \div 16 = \sim 113$ steps per epoch

কেন Batch? সব একসাথে নয় কেন? - ১,৮০৮টি ছবি GPU memory-তে একসাথে ধরে না - বড় batch-এ gradient noisier → generalization ভালো

Learning Rate (lr):

Model একটু ভুল করলে weight কতটুকু পরিবর্তন করবে?

lr = 0.1 → বড় পরিবর্তন (দ্রুত শেখে, কিন্তু overshoot করতে পারে)

lr = 0.001 → ছোট পরিবর্তন (ধীর, কিন্তু stable)

lr = 0.0001 → খুব ছোট (খুব ধীর, Transformer-এ ব্যবহার)

Analogy: পাহাড়ের নিচে নামছো (Loss কমাচ্ছে)। - বড় LR = বড় পদক্ষেপ → দ্রুত নামছো কিন্তু পড়ে যেতে পারো - ছোট LR = ছোট পদক্ষেপ → ধীরে কিন্তু নিরাপদ

● Overfitting ও Underfitting কী?

তিনটি scenario:

Scenario 1: Underfitting (খুব simple model)

Training accuracy: 60%, Test accuracy: 59%

Model কিছুই শিখতে পারেনি।

Scenario 2: Perfect fit (ideal)

Training accuracy: 88%, Test accuracy: 85%

Model general pattern শিখেছে।

Scenario 3: Overfitting (মুখস্থ করা)

Training accuracy: 98%, Test accuracy: 70%

Model training data memorize করেছে।

আমাদের case: Training ~90%+ → Validation ~72% → **Overfitting** আছে

কীভাবে বুঝবে Overfit? Training accuracy >> Validation accuracy → Overfit

Overfitting fix করার tools: 1. Dropout: random neuron-কে training-এ off করো 2. Weight Decay: বড় weight-কে penalize করো 3. Data Augmentation: আরও diverse training data 4. Early Stopping: validation performance না

● Precision, Recall, F1 কী — একদম সহজ ভাষায়

একটি doctor-এর উদাহরণ:

100 জন রোগী আসলো। - Actually অসুস্থ: 30 জন - Actually সুস্থ: 70 জন

Doctor সবাইকে দেখে বললো: - “অসুস্থ”: 40 জনকে - এর মধ্যে actually অসুস্থ: 25 জন ✓ (True Positive) - এর মধ্যে actually সুস্থ: 15 জন ✗ (False Positive) - “সুস্থ”: 60 জনকে - এর মধ্যে actually সুস্থ: 55 জন ✓ (True Negative) - এর মধ্যে actually অসুস্থ: 5 জন ✗ (False Negative)

Precision:

"Doctor যাদের অসুস্থ বলেছে, তাদের মধ্যে কতজন সত্যি অসুস্থ?"

$$\text{Precision} = 25 / (25+15) = 25/40 = 62.5\%$$

Recall:

"সত্যিকারের অসুস্থ 30 জনের মধ্যে Doctor কতজনকে ধরতে পেরেছে?"

$$\text{Recall} = 25 / (25+5) = 25/30 = 83.3\%$$

F1 Score:

"Precision ও Recall-এর balanced average"

$$F1 = 2 \times (62.5 \times 83.3) / (62.5 + 83.3) = 71.4\%$$

কখন Precision গুরুত্বপূর্ণ? Cancer screening: False Positive ঠিক আছে (extra test করবে), কিন্তু False Negative মানে real cancer miss করা — deadly! → Recall বেশি গুরুত্বপূর্ণ

কখন F1 গুরুত্বপূর্ণ? আমাদের project-এ imbalanced dataset → F1 Macro সবচেয়ে honest metric।

● Cross-Validation কী — সহজ ভাষায়

সমস্যা: মাত্র 1,808 image আছে।

যদি 80% train, 20% test করি: - Train: 1,446 image - Test: 362 image

এই 362 image-এ $\text{Fear} = 68 \times 0.2 = \sim 13$ image। 13টি image দিয়ে Fear-এর performance কি বিশ্বাসযোগ্য?

Cross-Validation-এর সমাধান:

সব 1,808 image-কে 5 ভাগে ভাগ করা।

Round 1: [ভাগ-1=Test] [ভাগ-2,3,4,5=Train] → F1 = 0.61

Round 2: [ভাগ-2=Test] [ভাগ-1,3,4,5=Train] → F1 = 0.63

Round 3: [ভাগ-3=Test] [ভাগ-1,2,4,5=Train] → F1 = 0.60

Round 4: [ভাগ-4=Test][ভাগ-1,2,3,5=Train] → F1 = 0.62

Round 5: [ভাগ-5=Test][ভাগ-1,2,3,4=Train] → F1 = 0.61

Final: Mean = 0.614, Std = 0.011

Report: 0.614 ± 0.011

এতে প্রতিটি image একবার test-এ আসে → আরও reliable estimate।

Stratified মানে: প্রতিটি ভাগে class distribution একই থাকে। - ভাগ 1-এ Fear: ~14 image, Joy: ~169 image - ভাগ 2-এ Fear: ~14 image, Joy: ~169 image - (প্রতিটি ভাগে সমান অনুপাত)

● Focal Loss কী — সহজ ভাষায়

Normal Cross-Entropy Loss-এর সমস্যা:

Model একটি Joy image দেখলো:

Prediction: Joy = 0.95 (95% confident, সঠিক)

Loss: $-\log(0.95) = 0.05$ → ছোট loss

Model একটি Fear image দেখলো:

Prediction: Joy = 0.85 (ভুল prediction, 85% confident)

Loss: $-\log(0.15) = 0.82$ → বড় loss

মোট training-এ: Joy image = 843টি, Fear = 68টি। Joy-এর loss সব মিলিয়ে: $843 \times 0.05 = 42.15$ Fear-এর loss সব মিলিয়ে: $68 \times 0.82 = 55.76$

Model Joy-এর জন্যই বেশি update হচ্ছে (কারণ Joy image বেশি) — **imbalance!**

Focal Loss-এর সমাধান:

Focal Loss = $(1 - p_{\text{correct}})^\gamma \times \text{Cross-Entropy}$
 $\gamma (\text{gamma}) = 2$

সহজ ভাষায়: - Easy example (model সঠিক, confident): Loss প্রায় শূন্য করে দাও - Hard example (model ভুল বা uncertain): Loss বড় রাখো

Joy image, correct, $p=0.95$:

$(1-0.95)^2 = 0.0025$ → loss প্রায় zero → model এখানে বেশি শিখবে না

Fear image, wrong, $p=0.15$:

$(1-0.15)^2 = 0.7225$ → loss বড় → model এখানে বেশি শিখবে

ফলে Fear class-এ model বেশি মনোযোগ দেয়!

● EMA (Exponential Moving Average) কী

সমস্যা: Training শেষের দিকে weights অনেক জাম্প করে।

Epoch 70: Validation F1 = 0.64
Epoch 71: Validation F1 = 0.61 ← Jump!
Epoch 72: Validation F1 = 0.65
Epoch 73: Validation F1 = 0.60 ← Jump!
Epoch 74: Validation F1 = 0.66

কোন epoch-এর weight ব্যবহার করবো?

EMA-এর সমাধান:

EMA Weight = $0.999 \times (\text{পুরনো EMA weight}) + 0.001 \times (\text{এই epoch-এর weight})$

এটা একটি **running average** — নতুন weight একটু একটু করে blend হয়।

Epoch 70: EMA = weighted average of 70 past epochs → smooth = 0.63

Epoch 71: EMA = weighted average → smooth = 0.635 (jump হয় না)

Analogy: তাপমাত্রা forecast। - আজকের তাপমাত্রা: 35°C (হঠাৎ গরম) - EMA: গত কয়েকদিনের weighted average = 32°C (stable estimate)

● Grad-CAM কী — সহজ ভাষায়

প্রশ্ন: Model “Anger” predict করলো — কিন্তু কোন অংশ দেখে?

Grad-CAM-এর কাজ:

Step 1: Image → Model → Prediction: “Anger”

Step 2: “Anger” score-কে backward propagate করা

Step 3: CNN-এর শেষ layer-এ gradient দেখা

(gradient মানে “এই pixel কতটা influence করলো?”)

Step 4: বড় gradient = বেশি influence = গুরুত্বপূর্ণ region

Step 5: Heatmap তৈরি → Original image-এ overlay

Result: একটি color-coded heatmap। - ● লাল: মডেল এখানে বেশি দেখেছে - ● নীল: মডেল এখানে কম দেখেছে

কেন গুরুত্বপূর্ণ?

ভালো result: Model ঠক ও চোখে লাল → সঠিক

খারাপ result: Model শার্টের color-এ লাল → model cheat করছে!

যদি model background দেখে emotion predict করে — সেটা spurious correlation। Grad-CAM সেটা ধরে ফেলে।

● Softmax কী এবং Probability কীভাবে আসে

Model-এর raw output (logits):

Anger: 2.1
Fear: -0.5
Joy: 3.8

Natural: 1.2
Sadness: 0.9
Surprise: -1.1

এগুলো probability নয় — raw scores! Softmax এগুলোকে probability-তে convert করে:

$\text{softmax}(x) = \exp(x) / \sum(\exp(\text{all } x))$

Anger: $\exp(2.1) / \text{total} = 8.17 / \dots = 0.19$
Fear: $\exp(-0.5) / \text{total} = 0.61 / \dots = 0.01$
Joy: $\exp(3.8) / \text{total} = 44.7 / \dots = 0.73$ ← সর্বোচ্চ!
Natural: $\exp(1.2) / \text{total} = 3.32 / \dots = 0.05$
Sadness: $\exp(0.9) / \text{total} = 2.46 / \dots = 0.04$
Surprise: $\exp(-1.1) / \text{total} = 0.33 / \dots = 0.006$
Sum: 1.00

Uncertainty Guardrail:

সর্বোচ্চ probability = 0.73 (Joy)
 $0.73 > 0.30$ (threshold) → confident prediction: "Joy"

কিন্তু যদি হতো:

Anger: 0.21, Fear: 0.18, Joy: 0.19, Natural: 0.15, Sadness: 0.14, Surprise: 0.13
সর্বোচ্চ = 0.21 < 0.30 → "Uncertain – refer to clinician"

● Batch Normalization কী

সমস্যা: Deep network-এ training-এর সময় layer-এর output-এর distribution পরিবর্তন হতে থাকে। Layer 5 যা expect করছে, Layer 4 কিন্তু অন্য ধরনের value দিচ্ছে। এটাকে **Internal Covariate Shift** বলে।

Batch Normalization-এর সমাধান:

প্রতিটি layer-এর output-কে normalize করা:

- Mean = 0 করা
- Standard Deviation = 1 করা
- তারপর learnable scale (γ) ও shift (β) দিয়ে adjust করা

Benefit: - Training অনেক stable হয় - বড় learning rate ব্যবহার করা যায় → দ্রুত training - কিছুটা regularization-এর কাজও করে

VGG-16 BN: “BN” মানে Batch Normalization আছে।

● Global Average Pooling (GAP) কী

CNN-এর শেষ convolutional layer output দেয়:

Feature Map: $7 \times 7 \times 512$
(7×7 spatial, 512 channels)

এটাকে কীভাবে classification-এ ব্যবহার করবো?

Option 1: Flatten: $7 \times 7 \times 512 = 25,088$ numbers $\rightarrow$ Fully Connected Layer

Option 2: Global Average Pooling (GAP): প্রতিটি 7×7 channel-এর average নাও $\rightarrow$ 512 numbers

GAP-এর সুবিধা: - কম parameters (overfitting কম) - Spatial information-এর summary - যেকোনো input size-এ কাজ করে

আমাদের Proposed Model-এ VGG16 stream $\rightarrow$ GAP $\rightarrow$ **512-d vector**।

● Squeeze-and-Excitation (SE) Block কী – বিস্তারিত

ধরো তোমার 512-d feature vector আছে। এই 512টি dimension বিভিন্ন তথ্য ধরে: - Dimension 1-50: চোখের information - Dimension 51-150: হাঁটের information - Dimension 151-300: নাকের information - Dimension 301-512: Background information

Emotion recognize করতে সব dimension সমান গুরুত্বপূর্ণ নয়।

Anger-এর জন্য: চোখ ও হাঁট বেশি গুরুত্বপূর্ণ Joy-এর জন্য: হাঁট (হাসি) বেশি গুরুত্বপূর্ণ

SE Block কাজ করে:

Step 1: Squeeze

512-d $\rightarrow$ FC Layer $\rightarrow$ 32-d (information compress করা)

Step 2: Excitation

32-d $\rightarrow$ FC Layer $\rightarrow$ 512-d $\rightarrow$ Sigmoid (0 থেকে 1)

Step 3: Scale

Original 512-d $\times$ Sigmoid output $\rightarrow$ গুরুত্বপূর্ণ channel amplify, অগুরুত্বপূর্ণ suppress

Analogy: একটি mixing board-এর মতো। - 512 টি channel = 512 টি knob - SE block শিখে নেয় কোন knob কতটা তুলতে হবে - Anger-এর জন্য: “চোখের channel” knob উঁচু, “background channel” knob নিচু

● Two-Stage Training কেন দরকার

একটি চাকরির interview analogy:

Stage 1 (160 epoch): General Learning

তুমি university-তে সব subject পড়লে।

(Model: সব data দেখে, সব class সমান সুযোগে, backbone সহ সব update)

এতে model general feature ভালো শিখলো।

Stage 2 (20 epoch): Specialized Fine-tuning

তুমি specific job-এর জন্য prepare করলে।
(Model: backbone frozen, শুধু classifier update, balanced sampling)
এতে model imbalanced class-এর জন্য specifically adjust করলো।

কেন Stage 1-এ balanced sampler নয়? যদি শুরু থেকে balanced করি, Joy = 843 $\rightarrow$ sample weight কম, Fear = 68 $\rightarrow$ weight বেশি। Model Fear image বারবার দেখে এবং Joy কম দেখে। Joy-এর performance কমে যায়!

কেন Stage 2-এ backbone frozen? Stage 1-এ backbone অনেক কষ্ট করে ভালো feature শিখেছে। Stage 2-এ যদি backbone-ও update হয়, balanced sampling-এর কারণে Joy features নষ্ট হয়ে যেতে পারে। Frozen backbone = protect করা হলো।

● Warmup Learning Rate কী

সমস্যা: Training শুরুতে weights random initialize করা থাকে। Random weights থেকে বড় LR দিয়ে শুরু করলে বিশৃঙ্খল update হয়।

Warmup:

Epoch 1: lr = 0.00001 (খুব ছোট)
Epoch 2: lr = 0.00002
Epoch 3: lr = 0.00004
...
Epoch 10: lr = 0.001 (পূর্ণ learning rate)
Epoch 11: lr = 0.00099 (cosine decay শুরু)
...
Epoch 160: lr = $\sim$ 0.00001 (প্রায় শূন্য)

Analogy: গাড়ি চালানো শেখার মতো। - প্রথমে slow speed-এ practice - তারপর normal speed - তারপর আস্তে আস্তে brake করে থামা

● dHash (Duplicate Detection) কীভাবে কাজ করে

সমস্যা: দুটো ছবি প্রায় একই কিন্তু pixel-exactly same নয়। (Different JPEG compression, slightly different crop)

dHash Algorithm:

Step 1: Image $\rightarrow$ Resize করো 9x8 (72 pixel)
Step 2: Grayscale করো (রঙ বাদ)
Step 3: প্রতিটি row-তে পাশের pixel-এর সাথে compare করো:
 বায়ের pixel > ডানের pixel? $\rightarrow$ 1
 বায়ের pixel $\leq$ ডানের pixel? $\rightarrow$ 0
Step 4: ফলাফল: 64-bit binary string (এটাই hash/fingerprint)

Image A hash: 1011001101...
Image B hash: 1011001101... (same $\rightarrow$ duplicate)
Image C hash: 1110110010... (different $\rightarrow$ unique)

Hamming Distance:

Hash A: 10110011

Hash B: 10110001

Difference: শুধু 7তম bit আলাদা → distance = 1

Distance ≤ 4 → near-duplicate → সরিয়ে দাও।

AdamW Optimizer কীভাবে কাজ করে

Gradient Descent (মূল idea):

নতুন weight = পুরনো weight - lr × gradient

Gradient বলে: “এই weight বাড়ালে loss বাড়ে নাকি কমে?”

Adam (Adaptive Moment Estimation): সব weight-এর জন্য একই lr ব্যবহার করা inefficient। - কিছু weight খুব sensitive (ছোট lr দরকার) - কিছু weight কম sensitive (বড় lr দেওয়া যায়)

Adam প্রতিটি weight-এর জন্য আলাদা lr রাখে:

m = average of recent gradients (momentum)

v = average of recent squared gradients (velocity)

effective_lr = lr / $\sqrt{v}$ × m

W = Weight Decay:

নতুন weight = পুরনো weight × (1 - weight_decay) - lr × gradient

Weight Decay প্রতিটি weight-কে একটু ছোট করে দেয় → বড় weight penalize।

কেন Weight Decay? বড় weight → model specific pattern-এ overfit। Penalty দিলে model simple, general weights prefer করে।

একটি Training Loop কীভাবে কাজ করে — Step by Step

```
for epoch in range(80):  
    for batch in dataloader:  
        # Step 1: Forward Pass  
        images, labels = batch  
        predictions = model(images) # model দিয়ে predict করা  
        # Step 2: Loss Calculate  
        loss = focal_loss(predictions, labels) # কতটা ভুল?  
        # Step 3: Backward Pass (Backpropagation)  
        optimizer.zero_grad() # পুরনো gradient মুছে দাও  
        loss.backward() # gradient calculate করে  
        # "কোন weight পরিবর্তন করলে loss কমবে?"  
        # Step 4: Weight Update  
        optimizer.step() # weight update করা
```

Step 5: EMA Update

```
ema.update(model)
```

smooth version update

Epoch শেষে Validation

```
val_f1 = evaluate(model, val_loader)
```

```
if val_f1 > best_f1:
```

```
    best_f1 = val_f1
```

```
    save_checkpoint(model)
```

সেটা model save করো

```
if no_improvement_for_15_epochs:
```

```
    break
```

Early Stopping

● Inference (Test Time) কীভাবে কাজ করে

Training শেষ। এখন নতুন ছবি দিলে কী হয়?

নতুন Image (224×224×3)

↓

Resize + Normalize (training-এর মতো)

↓

[TTA View 1: Original]

[TTA View 2: Flipped]

[TTA View 3: Rotated +5°]

[TTA View 4: Rotated -5°]

[TTA View 5: Scaled + Cropped]

↓

প্রতিটি view → Model → Softmax Probability

↓

5টি probability-র Average

↓

Uncertainty Check: max_prob > 0.30?

↓ হ্যাঁ

↓ না

argmax → Emotion label "Uncertain"

২৪. বাস্তব জীবনে এই Project ব্যবহার হলে কেমন দেখাবে

একটি কাল্পনিক Clinical Scenario

🏥 Autism Therapy Center, ঢাকা

Dr. রাহেলা একটি tablet নিয়ে Rafi-র (৭ বছর, ASD) কাছে বসলেন।

Rafi-কে একটি গল্প শোনানো হচ্ছে।

Camera → Image Capture (প্রতি সেকেন্ডে ৫ frame)

↓

Proposed Model (inference)

↓

"Joy: 0.73" → "Joy" ✓

↓

Screen-এ Dr. রাহেলার সামনে:

Emotion: JOY

